PLAYCARYS · CAPACITY PLANNING

Scalability and Actor Ratios

All four kinds of scalability, the one cause behind the two that bite, and a staffing model showing why moderators track players while admins track games.

14 AUGUST 2026 · MEASURED AGAINST THE WORKING TREE · 247 KB per page load / 15 games / 23 review events

01 The four kinds of scalability

"Scalable" covers four different problems. They are not the same, they are not solved by the same work, and fixing one does nothing for the others. Naming them separately is what makes it possible to say which one PlayCarys actually has.

Kind	The question it answers	Where PlayCarys stands today
01 Load	More people at the same time, without slowing down or falling over.	**BINDING.** One backend process holding all session and matchmaking state in its own memory. It cannot be run twice.
02 Data	More rows, without queries getting slower.	**BINDING.** Every table is a JavaScript array scanned with .find(). No index exists anywhere, because there is no database.
03 Feature	Adding a new game, role or delivery kind without rewriting what is there.	**HEALTHY.** The game-launch resolver and the 15-module backend layout are genuinely good. A new game is a data row, not a code change.
04 Cost	Revenue growing faster than the bill for serving it — including the wage bill.	**UNPRICED.** No revenue yet, and the staffing side is unmodelled. Sections 04 and 05 model it.

THE TWO BINDING ONES SHARE A SINGLE CAUSE

Every page loads the entire database into the browser, and every save posts the whole thing back. That one decision is why two tabs overwrite each other, why deleted games came back, and why accounts forked in two. The append-only unions, tombstones and merge logic accumulated since are all patches on it — and none of them make the payload smaller.

What it costs, measured today

247 KB	65%	86.7 KB	288 KB
DATABASE ON THE WIRE, PER PAGE LOAD	OF THAT IS THE GAMES TABLE ALONE	ONE GAME'S SCAN MANIFEST	OF JAVASCRIPT ALONGSIDE IT

The single worst offender is game 25, Ancient Beast, at a **99 KB row**. Not because of its cover image — because the security scan's per-file manifest, listing all 665 files in the bundle, is stored on the game record itself. Every player's browser downloads the complete file listing of every uploaded game, on every page, forever. Moving that one field off the row is the cheapest large win available.

The trajectory matters more than today's number

Matches recorded	Shipped per page load	What that feels like
96 (today)	22 KB	Fine.
1,000	0.2 MB	Noticeable on college wifi.
10,000	2.3 MB	Every page takes seconds. Phones struggle.
100,000	22.8 MB	Unusable. The tab runs out of memory.

A match row measures 239 bytes. You are roughly a hundred real players from the point where this stops being theoretical.

02 Actor ratios: what scales with what

There is no single staff-to-player ratio for this platform, and looking for one is the mistake. **The five actors sit on three different axes**, and only two of them are ratios at all.

Actor	Driven by	Is it a ratio?	Why
Player	Demand	It IS the denominator	Everything else is measured against this number.
Moderator	Players	Yes — linear in P	Reports are filed by people, at a roughly constant rate per person. Doubling players doubles the queue.
Admin	Games	Yes — linear in G	Their work is reviewing bundles. A platform with 10 games and a million players needs almost no admins.
Publisher	The market	No — recruited, not staffed	They are supply, not headcount. But they set G, so they are the upstream cause of admin load.
Superuser	Nothing	No — fixed at 1 or 2	A break-glass role, not a workforce. Adding more adds risk, not capacity. See section 05.

> **THIS IS THE WHOLE INSIGHT**
> Moderators scale with **players**. Admins scale with **games**. These are independent axes, so "how many staff per player" has no stable answer — it depends entirely on how big the catalogue is. Section 03 shows the same player count needing nearly nine times the staff purely because of catalogue size.

The moderator formula

Moderators = ceil(P x r x t / (60 x H)), with a floor of 2 for on-call coverage.
P = monthly active players, r = 0.03 reports per player per month, t = 12 minutes mean handling time, H = 56 productive hours per person per month.

Active players	Reports / month	Mod-hours / month	Moderators	Ratio
100	3	0.6	**1**	1 : 100
1,000	30	6.0	**1**	1 : 1,000
10,000	300	60.0	**2**	1 : 5,000
50,000	1,500	300.0	**6**	1 : 8,333
200,000	6,000	1,200.0	**22**	1 : 9,091

The report rate is a planning assumption, not a measurement — your live data has three reports from six seeded players, which is far too little to fit a rate to. Re-fit r from your own numbers after the first month of real traffic; the formula does not change, only that one constant does.

The admin formula

Review events = (new games x 2.3) + (live games x 0.33), then Admins = ceil(events x 35 / (60 x 56)).
The 2.3 is measured from your own data: 23 review events across 10 reviewed games. The re-review term exists because approval binds to approved_bundle_hash, so **every re-upload revokes approval** and sends the build back through review.

Live games	New / mo	Review events / mo	Admin-hours / mo	Admins	Publishers implied
15	2	9.6	5.6	**1**	~3
50	5	28.0	16.3	**1**	~9
200	15	100.5	58.6	**2**	~36
1,000	60	468.0	273.0	**5**	~182
5,000	250	2,225.0	1,297.9	**24**	~909

Publishers implied uses the measured 5.5 games per studio. This is the link between the "random" publisher count and the admin headcount: you do not provision publishers, but every one you recruit adds about 5.5 games, and those games are what you must staff for.

03 How the number of games changes the ratio

This is the question worth asking, because the answer is counter-intuitive. Hold the player count completely still at 10,000 and vary only the catalogue:

Live games	Moderators	Admins	Total staff	Staff-to-player ratio
15	**2**	**1**	**3**	1 : 3,333
200	**2**	**2**	**4**	1 : 2,500
1,000	**2**	**5**	**7**	1 : 1,429
5,000	**2**	**24**	**26**	1 : 385

> **THE ANSWER, IN ONE LINE**
> Same 10,000 players throughout. Staff goes from 3 to 26 — **8.7 times the headcount for the identical audience**, purely because the catalogue grew. The staff-to-player ratio moves from 1:3,333 to 1:385. If you ever quote a single ratio for PlayCarys, it is wrong unless you also state the catalogue size.

The mirror image: hold games still, vary players

Active players	Moderators	Admins	Total staff
1,000	**1**	**2**	**3**
10,000	**2**	**2**	**4**
50,000	**6**	**2**	**8**
200,000	**22**	**2**	**24**

A 200-fold increase in players, and the admin count does not move at all. That is the cleanest possible demonstration that the two roles are not on the same axis — and the reason a single "actor ratio" for the platform cannot exist.

The second-order effect

More games does nudge moderator load upward, because more games means more sessions per player and therefore slightly more surface for reports. But the report rate per *player* stays roughly flat, so this is small next to the direct effect on admins. Plan moderators on players; plan admins on games; treat the cross-term as noise until you can measure it.

04 The lever that changes the slope

The admin-to-games ratio is not a law of nature. It is set by one number: **the share of uploaded bundles that clear review without a human**. Today that share is zero, and your own scan calibration note says why.

scan.service.ts is deliberately conservative — "fail" is reserved for certainties, so HexGL, 2048, Hextris and Ancient Beast all return **warn**, never pass and never fail. A warn is not actionable automatically, which means **every single bundle needs a person**. Here is what those people are reading:

Game	Files in bundle	Findings to read	Scan verdict	Review events it took
HexGL	197	17	warn	1
2048	34	4	warn	1
Hextris	73	23	warn	1
Ancient Beast	665	27	warn	13

Note the tail. Nine games took one or two review events each; Ancient Beast took thirteen. Admin load per game is not a constant — it is a mean of 2.3 with a long right tail, and the tail is driven by bundle size and by re-uploads.

What raising the auto-clear rate is worth

At a 1,000-game catalogue taking 60 new submissions a month:

Bundles auto-cleared	Events needing a human	Admin-hours / mo	Admins	Versus today
0%	468	273	**5**	—
30%	328	191	**4**	1 fewer people
60%	187	109	**2**	3 fewer people
80%	94	55	**1**	4 fewer people

THE EQUIVALENT LEVER FOR MODERATORS

Two of your three seeded reports name the same player. That is not a coincidence in the seed data — it is how moderation actually distributes: a small number of accounts generate most of the queue. Auto-flagging repeat subjects, and auto-grouping duplicate reports about one session, cuts the queue without cutting the standard. Everything else about the moderator formula stays the same; only t, the handling time, comes down.

05 Publishers and superusers: why they are not ratios

Publishers are supply, not headcount.

You do not provision publishers the way you provision moderators — you recruit them, and how many you get is set by the market, not by a formula. In that sense they are genuinely "random", as you put it. But they are not disconnected from the model: each publisher brings roughly 5.5 games, and games are the thing that sets admin headcount. So a publisher is not a staffing cost *directly*; it is a staffing cost one step removed. Recruiting twenty studios is a decision to hire admins, whether or not it is framed that way at the time.

Superusers are a fixed count, forever.

One, or two for the case where the first is unreachable. Never more, and never scaled with anything. The reason is in your own RolesGuard: a verified superuser token returns true before any required-role check runs, so a superuser bypasses every permission in the platform. Each additional superuser is not additional capacity — it is an additional full-platform compromise vector. When someone needs more power, the correct answer is to add a narrower role, not another superuser.

A CONSEQUENCE WORTH PLANNING FOR NOW

With one superuser and no password-reset flow, losing that account loses the platform. That is not a scaling problem, it is a single point of failure that already exists at your current size — and it is the same missing flow that appears at the top of the revenue report's sequence.

Reference: a worked staffing plan

Each row is what you would actually hire at that stage.

Stage	Players	Games	Mods / Admins	Pubs	What it looks like
Today	13	15	**1 / 1**	~3	Founder does everything. The seeded 2 mods / 2 admins are theatre.
First 1k	1,000	40	**1 / 1**	~7	One part-time moderator. You still review every game yourself.
10k players	10,000	200	**2 / 2**	~36	Two moderators for coverage, two admins. Automation not yet urgent.
50k players	50,000	600	**6 / 4**	~109	Six moderators, three admins. Auto-clear becomes worth building here.
200k players	200,000	2,000	**22 / 10**	~364	Twenty-two moderators, six admins. Without auto-clear, ten.

06 The technical roadmap

Four phases in dependency order. Each is independently useful — you can stop after any one of them and be better off than before.

01 Stop shipping the whole database

No new technology, no migration, and the biggest immediate win. Three moves: get scan.manifest off the game row; paginate match history instead of loading all of it; and convert pages one at a time from the whole-database sync to the per-endpoint API calls that already exist.

The player dashboard already works this way, so this is a pattern to copy rather than invent. It also closes Review-4 gap 6, which asks for exactly this.

`EFFORT: DAYS. REMOVES ~65% OF PAGE WEIGHT ON ITS OWN.`

02 PostgreSQL behind Prisma

Keep every table name and column. The schema is good — twelve tables, real keys, proper IS-A specialisation, two honest junctions — it simply needs an engine that enforces it. Prisma can generate the migration from the shape you already have.

The moment it lands, whole classes of bug become unwritable: the forked account, the resurrected game, the row owned by the string "undefined". Do it after phase 1, not before — migrating while the browser still holds the whole database means buying constraints and keeping the thing that violates them.

`EFFORT: A WEEK. THIS IS THE DATA-SCALABILITY FIX.`

03 Make the backend stateless

Sessions, the matchmaking queue and the socket rooms all live in the memory of one Node process. That is why a second instance cannot be run, and why the whole platform dies with a single crash. Move that state into Redis and the process becomes disposable.

Then four of them can run behind a load balancer and losing any one costs nothing. Note that front-end/server.js also binds port 3000 and collides with the backend — worth deleting or repointing before this phase.

`EFFORT: A WEEK. THIS IS THE STEP THAT ACTUALLY MEANS "SCALABLE".`

04 Game bundles off the app server

Uploaded builds are served by the same process that runs the API, from the same disk. One popular game and the API slows down because it is busy shipping textures. Move front-end/games/ to object storage behind a CDN.

This also closes the security gap already documented in the code: bundles currently run **same-origin** inside play.html, so an uploaded game can read a player's localStorage. A separate origin fixes the performance problem and that hole in one move.

`EFFORT: DAYS. FIXES A KNOWN SECURITY GAP AS A SIDE EFFECT.`

What not to build yet

Microservices, Kubernetes, message queues, database sharding. Every one of them solves a problem that does not exist below roughly a hundred thousand users, and each adds a failure mode that has to be debugged by one person at two in the morning. Four Node processes, one Postgres and one Redis will carry PlayCarys considerably further than it is likely to need to go.

> **HOW THE TWO REPORTS FIT TOGETHER**
> Cost scalability — the fourth kind, and the one usually left unmodelled — is where these two documents meet. The revenue report says what comes in per player and per game; this one says what a player and a game each cost to serve and to staff. A platform is viable when revenue per game exceeds the review-and-moderation cost of carrying that game. That is the number to watch, and neither document can produce it alone.